How the Smart City Platform Was Made

1. Architecture Overview

The AI Smart City Backend is built using a modern Python Flask framework for the backend API, combined with Vanilla JavaScript, HTML5, and CSS3 for the responsive dashboard. The database uses SQLite with SQLAlchemy as the ORM to store location data, traffic logs, complaints, and environmental metrics.

2. Backend & Blueprints

The backend is modularized into 'Blueprints' representing different domains: Traffic, Waste, Complaints, Predictions, and Environment. A background simulator job continuously generates simulated telemetry to mimic a live city environment.

3. AI & Predictions

The platform integrates basic AI models to forecast traffic congestion and waste bin fill levels. An integrated AI assistant (using external APIs) can process natural language queries from citizens and return real-time city data.

4. Frontend Interface

The frontend consists of an Admin Dashboard (index.html) and a Citizen Portal (citizen.html). We heavily utilized CSS variables to implement a dynamic Light/Dark mode. The map integration was built using Leaflet.js, mapping traffic zones and rendering congestion tooltips.

5. Development Workflow

Development progressed from structuring the database schema, building out the REST APIs, implementing the background simulation, to styling the frontend dashboards. Bugs in the JavaScript Promises and null reference exceptions were resolved to create a stable, asynchronous frontend.